

2019-2020学年秋季学期

漏洞利用与攻防实践

*Exploiting Software Vulnerability-
Techniques and Practice*

报告：何玮

演示：陈沛茹

漏洞利用与攻防实践

Exploiting Software Vulnerability-Techniques and Practice

[第九组] 模糊测试实践之变异策略

概要

- AFL_2.52b
- T-Fuzz: fuzzing by program transformation
- REDQUEEN: Fuzzing with Input-to-State Correspondence
- FAIRFUZZ: A Targeted Mutation Strategy for Increasing Greybox Fuzz Testing Coverage
- 实验

fuzz_one()

- afl-fuzz.c line 4936 ~ 6621
- 判断是否跳过 fuzz_one()
- 映射 testcase 到内存
- calibrate_case() : 校准 testcase
- trim_case() : 修剪 testcase
- calculate_score() : 给 testcase 打分调整 havoc 阶段的用时
- deterministic stage : 确定性变异阶段
- havoc stage : 随机性变异阶段 (大破坏)
- splicing : 拼接 testcase
- 结束

calculate_score()

- afl_fuzz.c line 4679 ~ 4747
- 目的：调整 havoc 变异阶段的用时
- 返回值为分数 perf_score，初始为100
- 执行时间越短（0.1x~4x）perf_score 值越大（10~150）
- bitmap_size 越大（0.3x~3x）perf_score 系数越大（0.25x~3x）
- 发现路径越新，perf_score 系数越大（1x~4x）
- 路径深度越深，perf_score 系数越大（1x~5x）
- perf_score 最大值 $HAVOC_MAX_MULT \times 100$

deterministic

- afl_fuzz.c line 5091 ~ 5419 (BitFlip)
- bitflip 1/1 : 每次翻转 1bit , 按 1bit 步长从头开始。同时检测 token
- bitflip 2/1 : 每次翻转相邻的 2bit , 按 1bit 步长从头开始
- bitflip 4/1 : 每次翻转相邻的 4bit , 按 1bit 步长从头开始
- bitflip 8/8 : 每次翻转相邻的 8bit (1B) , 按 8bit (1B) 步长从头开始 , 同时生成 effector_map
- bitflip 16/8 : 每次翻转相邻的 16bit (2B) , 按 8bit (1B) 步长从头开始
- bitflip 32/8 : 每次翻转相邻的 32bit (4B) , 按 8bit (1B) 步长从头开始

deterministic

- afl_fuzz.c line 5420 ~ 5677 (Arithmetic)
- arith 8/8 : 每次对 8bit (1B) 进行加减运算 , 按 8bit (1B) 步长从头开始
- arith 16/8 : 每次对 16bit (2B) 进行加减运算 , 按 8bit (1B) 步长从头开始
- arith 32/8 : 每次对 32bit (4B) 进行加减运算 , 按 8bit (1B) 步长从头开始
- 针对大小端都进行变异
- 跳过变异

deterministic

- afl-fuzz.c line 5678 ~ 5871 (Interesting value)
- interest 8/8 : 每次对 8bit (1B) 进行替换 , 按 8bit (1B) 步长从头开始
- interest 16/8 : 每次对 8bit (2B) 进行替换 , 按 8bit (1B) 步长从头开始
- interest 32/8 : 每次对 8bit (4B) 进行替换 , 按 8bit (1B) 步长从头开始
- INTERESTING_8 : -128、 -1、 0、 1、 16、 32、 64、 100、 127
- INTERESTING_16 : -32768、 -129、 128、 255、 256、 512、 1000、 1024、 4096、 32767
- INTERESTING_32 : -2147483648、 -100663046、 -32769、 32768、 65535、 65536、 100663045、 2147483647

deterministic

- afl-fuzz.c line 5872 ~ 6037 (dictionary)
- user extras(over) : 使用用户提供的 tokens 替换 , 从头开始
- user extras(insert) : 插入用户提供的 tokens , 从头开始
- auto extras(over) : 使用自动检测的 tokens 替换 , 从头开始

havoc

- afl-fuzz.c line 6046 ~ 6503
- case 0 : 随机翻转某 bit
- case 1 : 随机选取某 byte 设置为随机的 interesting value
- case 2 : 随机选取某 word 和大小端序设置为随机的 interesting value
- case 3 : 随机选取某 dword 和大小端序设置为随机的 interesting value

havoc

- case 4 : 随机选取某 byte 减去一个随机数
- case 5 : 随机选取某 byte 加上一个随机数
- case 6 : 随机选取某 word 和大小端序减去一个随机数
- case 7 : 随机选取某 word 和大小端序加上一个随机数
- case 8 : 随机选取某 dword 和大小端序减去一个随机数
- case 9 : 随机选取某 dword 和大小端序加上一个随机数
- case 10 : 随机选取某 byte 使用 XOR 设置为随机数
- case 11 ~ 12 : 随机删去某段 bytes

havoc

- case 13 : 随机选取某个位置插入一段随机长度内容
- case 14 : 随机选取某个位置替换为一段随机长度内容
- case 15 : 随机选取某个位置替换为自动生成或用户输入的 token
- case 16 : 随机选取某个位置插入自动生成或用户输入的 token

splicing

- afl-fuzz.c line 6504 ~ 6594
- 将当前的输入和一个随机选择的输入拼接得到新的输入，对新的输入执行 havoc
- 如果随机选择的输入和当前输入差别不大（只是一个 byte 的差别），则重新选择
- 如果随机选择的输入和当前输入差别比较大，则随机选取位置将两者分割为头部和尾部，将当前输入头部和随机选择的输入的尾部拼接

局限性

Header(4)	Keys(95)	Len(4)	Data(Len)	CRC(4)
-----------	----------	--------	-----------	--------

- Header(4) : 文件头 , 包含硬编码的魔数“SECO”
- Keys(95) : 声明 95 个唯一的字符 , 其 ASCII 值必须在 [32, 126] 的范围内
- Len(4) : 指定以下压缩数据 (Data) 的长度
- CRC(4) : CRC 校验和字段

局限性

Header(4)	Keys(95)	Len(4)	Data(Len)	CRC(4)
-----------	----------	--------	-----------	--------

- C1 : 检查魔数是否为“SECO”
- C2 : 检查 Keys 字段的范围和唯一性
- C3 : 检查 CRC 字段以确定是否有数据损坏
- decompress() : 存在一个栈缓冲区溢出漏洞

```
1
2 #define KEY_SIZE 95
3 int sc_decompress(int infd, int outfd) {
4     unsigned char keys[KEY_SIZE];
5     unsigned char data[KEY_SIZE];
6     char *header = read_header(infd)
7     // C1: check for hardcoded values
8     if (strcmp(header, "SECO") != 0)
9         return ERROR;
10    read(infd, keys, KEY_SIZE);
11    memset(data, 0, sizeof(data));
12    // C2: range check and duplicate check for keys
13    for (int i = 0; i < sizeof(data); ++i) {
14        if (keys[i] < 32 || keys[i] > 126)
15            return ERROR;
16        if (data[keys[i] - 32]++ > 0)
17            return ERROR;
18    }
19    unsigned int in_len = read_len(infd);
20    char *in = (char *) malloc(in_len);
21    read(infd, in, in_len);
22    unsigned int crc = read_checksum(infd);
23    // C3: check the crc of the input
24    if (crc != compute_crc(in, in_len)) {
25        free(in);
26        return ERROR;
27    }
28    char *out;
29    unsigned int out_len;
30    // Bug: function with stack buffer overflow
31    decompress(in, in_len, keys, &out, &out_len);
32    write(outfd, out, out_len);
33    return SUCCESS;
34 }
```

局限性

Header(4)	Keys(95)	Len(4)	Data(Len)	CRC(4)
-----------	----------	--------	-----------	--------

- C1：检查魔数是否为“SECO”
- C2：检查 Keys 字段的范围和唯一性
- C3：检查 CRC 字段以确定是否有数据损坏
- decompress()：存在一个栈缓冲区溢出漏洞

难以绕过完整性检查

```
1
2 #define KEY_SIZE 95
3 int sc_decompress(int infd, int outfd) {
4     unsigned char keys[KEY_SIZE];
5     unsigned char data[KEY_SIZE];
6     char *header = read_header(infd)
7     // C1: check for hardcoded values
8     if (strcmp(header, "SECO") != 0)
9         return ERROR;
10    read(infd, keys, KEY_SIZE);
11    memset(data, 0, sizeof(data));
12    // C2: range check and duplicate check for keys
13    for (int i = 0; i < sizeof(data); ++i) {
14        if (keys[i] < 32 || keys[i] > 126)
15            return ERROR;
16        if (data[keys[i] - 32]++ > 0)
17            return ERROR;
18    }
19    unsigned int in_len = read_len(infd);
20    char *in = (char *) malloc(in_len);
21    read(infd, in, in_len);
22    unsigned int crc = read_checksum(infd);
23    // C3: check the crc of the input
24    if (crc != compute_crc(in, in_len)) {
25        free(in);
26        return ERROR;
27    }
28    // ... rest of the function ...
29    // ...
30    // ...
31    // ...
32    // ...
33    return SUCCESS;
34 }
```

观察

- 完整性检查可以分为 NCC (Non-Critical Check) 和 CC (Critical Check)
- 可以删除 NCC 而不会误报 BUG
- 去掉 NCC 后发现的 BUG 能在原程序中复现
- 去掉 CC 后可能引入假 BUG , 这些 BUG 不能在原程序复现

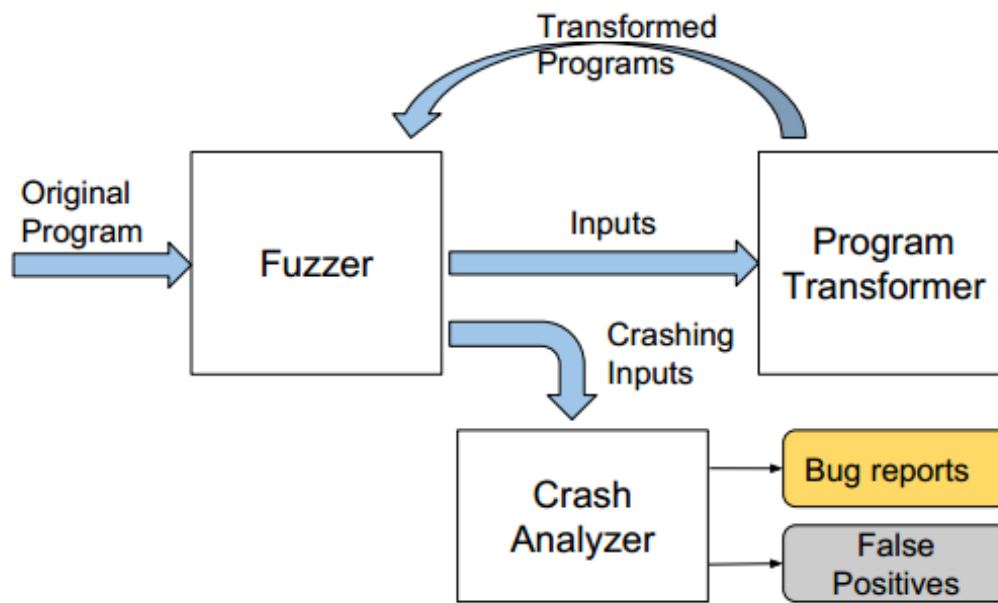
流程

Algorithm 1: Fuzzing with T-Fuzz

Input: *program*: original program

```
1 programs  $\leftarrow$  {program}  
2 while programs  $\neq \emptyset$  do  
3    $p \leftarrow \text{Choose\_Program}(\textit{programs})$   
4    $\textit{inputs} \leftarrow \text{Fuzzer}(p)$   
5    $\textit{programs} \leftarrow$   
    $\textit{programs} \cup \text{Program\_Transformer}(p, \textit{inputs})$ 
```

结构



- Fuzzer : AFL 或 honggfuzz
- Program Transformer : 生成修改后的程序
- Crash Analyzer : 过滤虚假的 BUG

检测候选NCC

- 背景知识：完整性检查被编译为程序中的跳转指令，在CFG中表示为具有两个输出边（T或F）的基本块S
- 使用CFG中所有的边界边（boundary edge）近似作为候选NCC

Algorithm 2: Detecting NCC candidates

Input: *program*: The binary program to be analyzed

Input: *CE*: cumulative edge coverage

Input: *CN*: cumulative node coverage

1 $cfg \leftarrow CFG(program)$

2 $NCC \leftarrow \emptyset$

3 **for** $e \in cfg.edges$ **do**

4 **if** $e \notin CE \wedge e.source \in CN$ **then**

5 $NCC \leftarrow NCC \cup \{e\}$

Output: *NCC*: detected NCC candidates

检测候选NCC

○计算累积边覆盖信息和结点覆盖信息

Algorithm 3: Calculating cumulative edge and node coverage

Input: *inputs*: inputs to collect cumulative coverage

```
1  $CE \leftarrow \emptyset$ 
2  $CN \leftarrow \emptyset$ 
3 for  $i \in inputs$  do
4    $trace \leftarrow dynamic\_trace(i)$ 
5   for  $e \in trace$  do
6      $CE \leftarrow CE \cup \{e\}$ 
7      $CN \leftarrow CN \cup \{(e.source, e.destination)\}$ 
```

Output: CE : cumulative edge coverage

Output: CN : cumulative node coverage

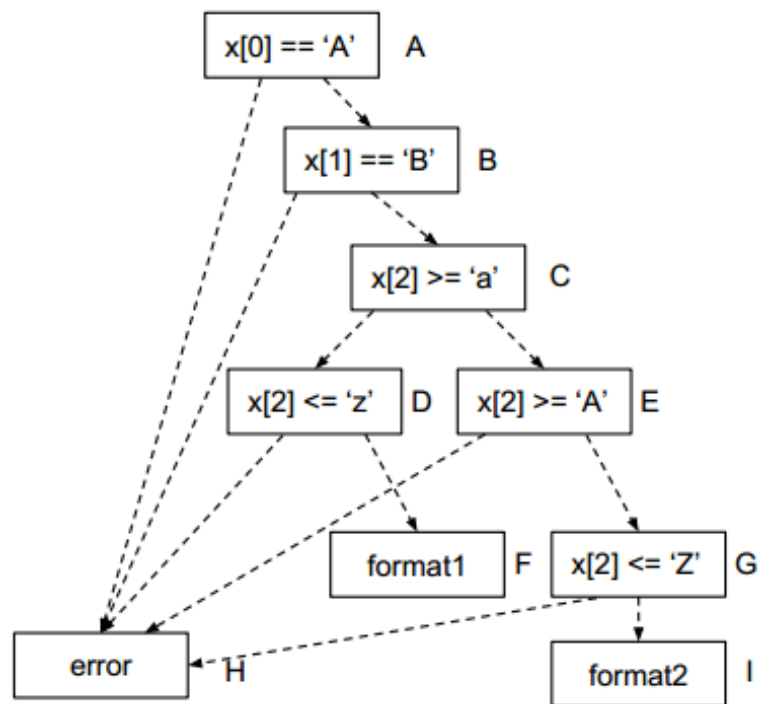
检测候选NCC

```

void main() {
    char x[16];
    read(stdin, x, 16);

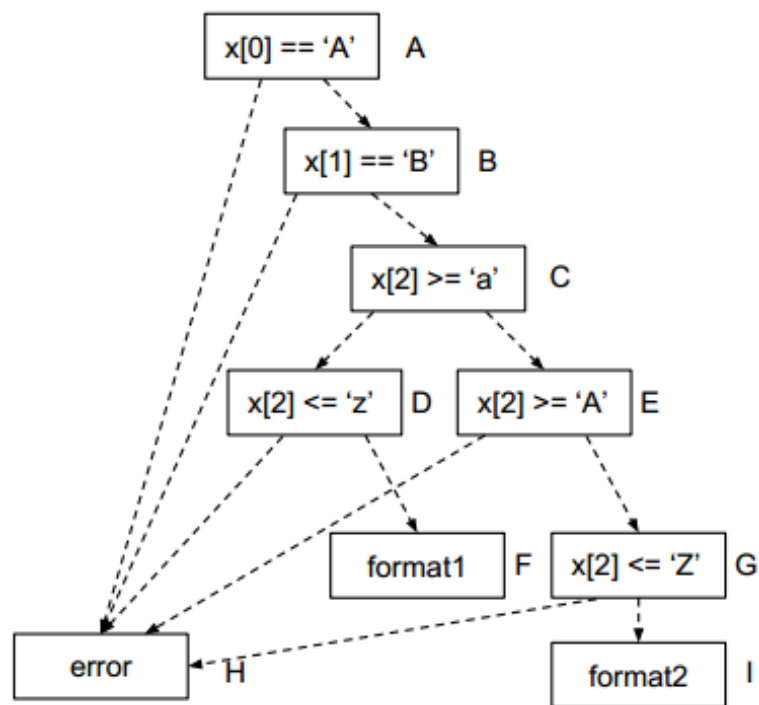
    if (x[0] == 'A' && x[1] == 'B') {
        if (x[2] >= 'a') {
            if (x[2] <= 'z') {
                format1(x);
            } else {
                goto failure;
            }
        } else {
            if (x[2] >= 'A' && x[2] <= 'Z')
                format2(x);
            else {
                goto failure;
            }
        }
    }
    failure:
    error();
}

```



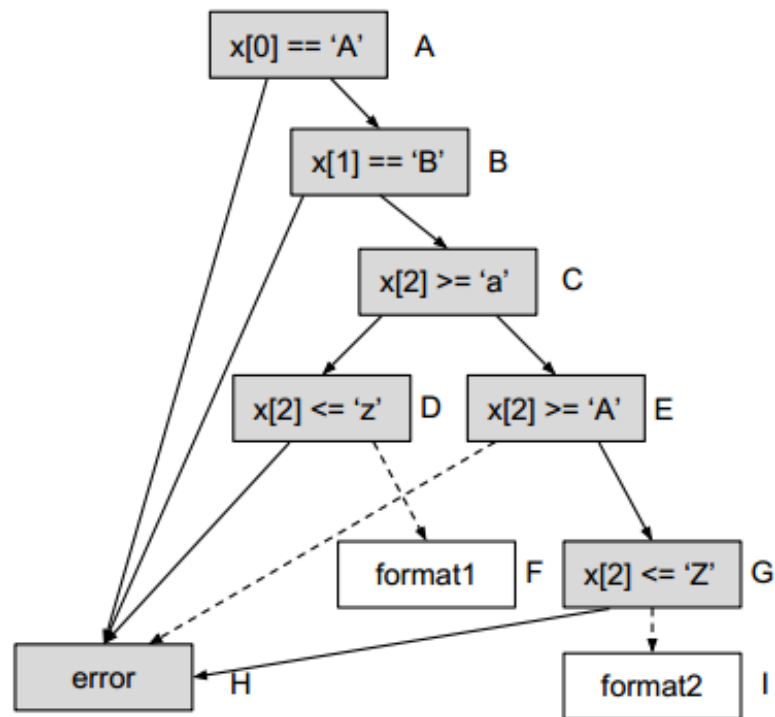
检测候选NCC

- 假设输入 {"123...", "A12...", "AB[...", "AB{..."}
- 123... : A->H
- A12... : A->B->H
- AB[... : A->B->C->E->G->H
- AB{... : A->B->C->D->H
- 注 : $\text{int}([' = 91$ $\text{int}([' = 123$



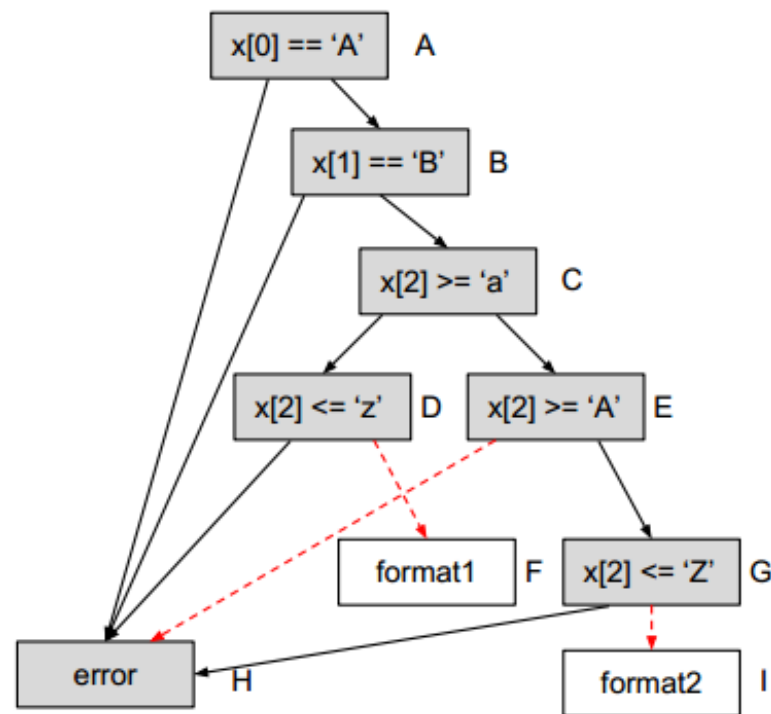
检测候选NCC

- 假设输入 {"123...", "A12...", "AB[...", "AB{..."}
- 123... : A->H
- A12... : A->B->H
- AB[... : A->B->C->E->G->H
- AB{... : A->B->C->D->H
- 注 : $\text{int}([' = 91$ $\text{int}([' = 123$



检测候选NCC

- 假设输入 {"123...", "A12...", "AB[...", "AB{..."}
 - 123... : A->H
 - A12... : A->B->H
 - AB[... : A->B->C->E->G->H
 - AB{... : A->B->C->D->H
- 注 : $\text{int}([' = 91$ $\text{int}([' = 123$
- 检测到的候选 NCC 为 **D->F**、**E->H**、**G->I**



修剪NCC

- 修剪不在可执行文件或特定库中的所有候选 NCC
- 修剪对象为立即终止程序的错误代码的 NCC

```
void main() {  
    char x[10];  
  
    if (read(0, x, 10) == -1)  
        goto failure;  
    // main logic for processing x  
    ...  
    return;  
failure:  
    error();  
}
```

转换程序

○使用否定的条件跳转替换检测到的候选 NCC

Algorithm 4: Transforming program

Input: *program*: the binary program to transform
Input: *c_addrs*: the addresses of conditional jumps negated in the input program
Input: *NCC*: NCC candidates to remove

```

1 transformed_program  $\leftarrow$  Copy(program)
2 for  $e \in NCC$  do
3   basic_block  $\leftarrow$ 
     BasicBlock(transformed_program, e.source)
4   for  $i \in \text{basic\_block}$  do
5     if  $i$  is a conditional jump instruction and
        $i.addr \notin c\_addrs$  then
6       negate_conditional_jump(program, i.addr)
7        $c\_addrs \leftarrow c\_addrs \cup \{i.addr\}$ 
8       break

```

Output: *transformed_program*: the generated program with NCC candidate disabled
Output: *c_addrs*: the locations modified in the transformed program

过滤误报并复现错误

Algorithm 5: Process to filter out false positives

Input: *transformed_program*: the transformed program

Input: *c_addrs*: addresses of negated conditional jumps

Input: *input*: the crashing input

Input: *CA*: the crashing address

```

1 PC  $\leftarrow$  make_constraint(input)
2 CT  $\leftarrow$  PC
3 CO  $\leftarrow$   $\emptyset$ 
4 TC, addr  $\leftarrow$ 
   preconstraint_trace(transformed_program, CT, entry)
5 while addr  $\neq$  CA do
6   if addr  $\in$  c_addrs then
7     CO  $\leftarrow$  CO  $\cup$   $\neg$ TC
8   else
9     CO  $\leftarrow$  CO  $\cup$  TC
10    TC, addr  $\leftarrow$ 
       preconstraint_trace(transformed_program, CT, i)
11 CO  $\leftarrow$  CO  $\cup$  extract_crashing_condition(TC)
12 result  $\leftarrow$  SAT(CO)
Output: result: A boolean indicating whether input is a
           false positive
Output: CO: The set of constraints for generating the
           inputs in the original program

```

过滤误报并复现错误

- L5→L6 标记为候选 NCC
- 转换程序否定 x 的检查
- $\neg(x \neq 0xdeadbeef)$ 和 $y == \text{crash_value}$ 被添加到 CO
- $\{\neg(x \neq 0xdeadbeef); y == \text{crash_value}\}$ 可满足, 则 crash 为原始程序真正错误

```
void main() {  
    int x, y;  
    read(0, &x, sizeof(x));  
    read(0, &y, sizeof(y));  
    if (x == 0xdeadbeef)  
        *(int *)y = 0;  
}
```

过滤误报并复现错误

○ L6→L7 标记为候选 NCC

○ 转换程序否定 index 的检查

○ $\neg(\text{index} < 0 \parallel \text{index} > 3)$ 和 $\text{index} == \text{crash_value}$ (例如 10) 被添加到 CO

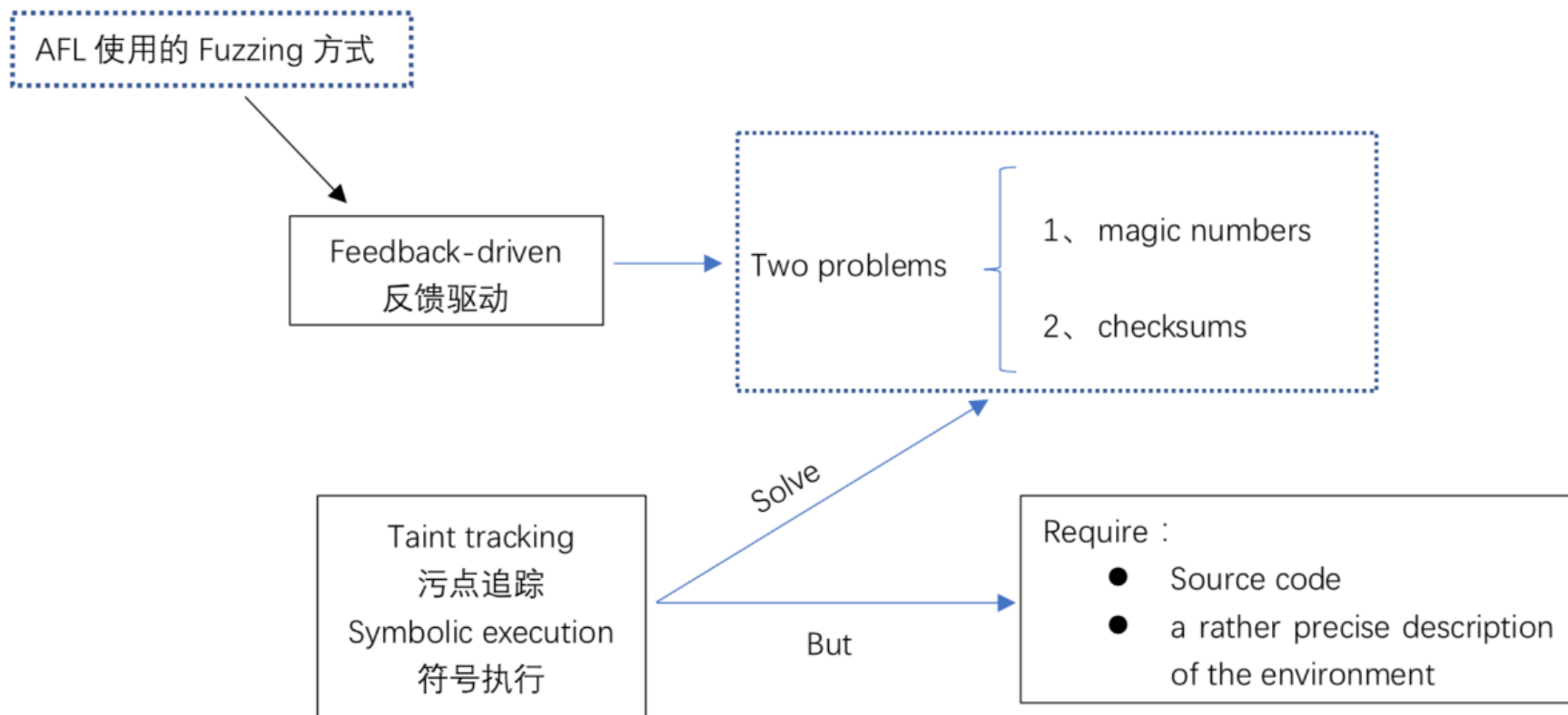
○ $\{\neg(\text{index} < 0 \parallel \text{index} > 3); \text{index} == 10\}$ 不可满足, 则 crash 为误报

```
void main() {  
    char secrets[4] = "DEAD";  
    int index;  
    read(0, &index, sizeof(index));  
  
    if (index >= 0 && index <= 3)  
        output(secrets[index]);  
}
```

效果

- 在CGC数据集中，T-Fuzz在166个二进制文件中发现了真实的错误，比与Driller多45个，比AFL多61个。
- 在LAVA-M数据集中，当目标程序中存在硬件检查（如校验和）时，T-Fuzz显示出明显更好的性能。
- 评估实际程序时发现了3个新BUG：两个在magick中，一个在pdftohtml中

T-Fuzz局限性

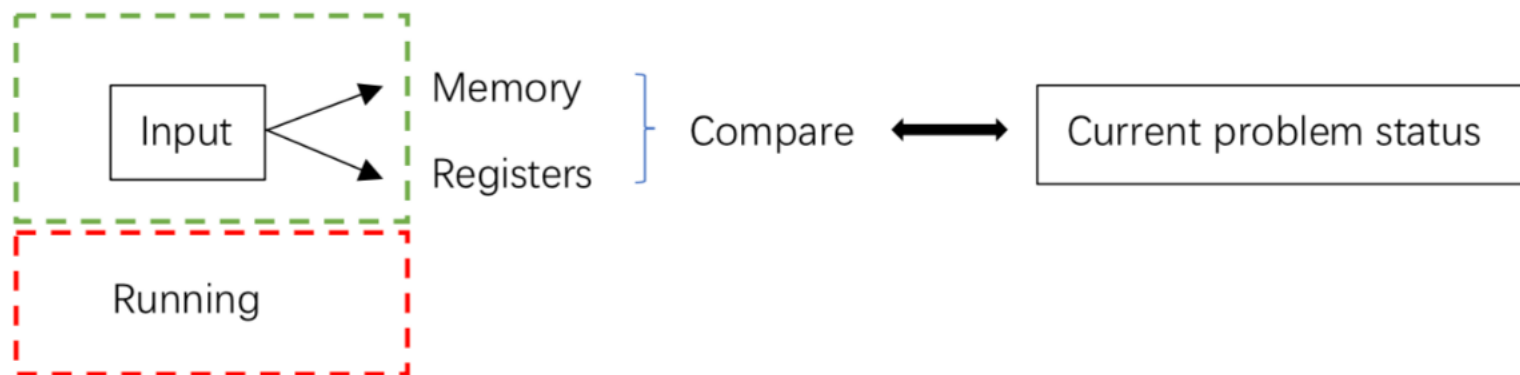


观察

- 在运行时，部分输入会直接传入到 memory（内存）或 registers（寄存器）中进行比较
- 则输入与当前 status 之间存在极强的 input-to-status 关联性

思路

- 首先在程序执行时识别并追踪比较指令
- 然后判断特定位置 input 的修改是否会对程序 status (memory , registers) 的修改
- 最后通过修改 fuzzing 中的输入 , 来确定修改是否给 fuzzing 带来了新的 code coverage



Input-to-status

处理魔数字节

```
if (u64(input) == u64("MAGICHDR"))
    bug(1);
```

C-Code		$u64(input) == u64("MAGICHDR")$
Input		"TestSeedInput"
Observed (ASCII)		"deeStesT" == "RDHCIGAM"
Variations for < and > comparisons		"deeStesT" "RDHCIGAL" "deeStesT" "RDHCIGAN"
Mutations after little- endian encoding		<"TestSeed" $\mapsto$ "MAGICHDR"> <"TestSeed" $\mapsto$ "LAGICHDR"> <"TestSeed" $\mapsto$ "NAGICHDR">

追踪

- 在 fuzz 一个新的输入时，进行一次 tracing
- hook 所有的比较指令（包括一些 switch 结构、比较函数、编译器发出的指令）并提取参数
- 例：将 “TestSeedInput” 作为上述代码的输入。比较指令检查输入中的前 8 个字节（u64）是否等于对字符串 “MAGICHDR”（u64）。由于整数通常以小尾数格式进行编码，因此最终追踪得到 cmp “deeStseT”，
“RDHCIGAM”

变化

- 在运行时，我们不知道比较之后检查了哪些标志，也无法区分不同比较操作（如小于和等于）
- 将一些例如加 1 或减 1 的变化应用于比较值，增加了大小差一（ off-the-one ）错误的触发几率
- 例：将加 1 或减 1 应用在“RDHCIGAM”上得到“RDHCIGAL”和“RDHCIGAN”

编码

- 在达到理想比较效果之前，很可能以不同的方式处理了输入
- 为了处理输入解码最常见的情况，创建更多的候选变异，对突变应用不同编码，主要包括大小端变换、零 / 算术扩展、反向、C字符串化（尾零）、Memory(n)、ASCII 码编码
- 例：对当前的“RDHCIGAM”、“RDHCIGAL”和“RDHCIGAN”应用小端编码得到“MAGICHDR”、“LAGICHDR”和“NAGICHDR”

应用

- 创建变异规则 `< pattern -> repl >` 标识输入中要用变异 repl 替换的部分
- 可应用于原子比较而无需进一步修改或 hook 目标
- 大大减少尝试替换的候选位置的数量
- 例：输入 “TestSeedInput” 的子字符串 “TestSeed” 与 “MAGICHDR” 进行比较，则仅用变异规则 `< TestSeed -> MAGICHDE >` 替换这部分，产生新的测试样例 “MAGICHDRInput”、“LAGICHDRInput”、“NAGICHDRInput”

- 40

字符串和内存

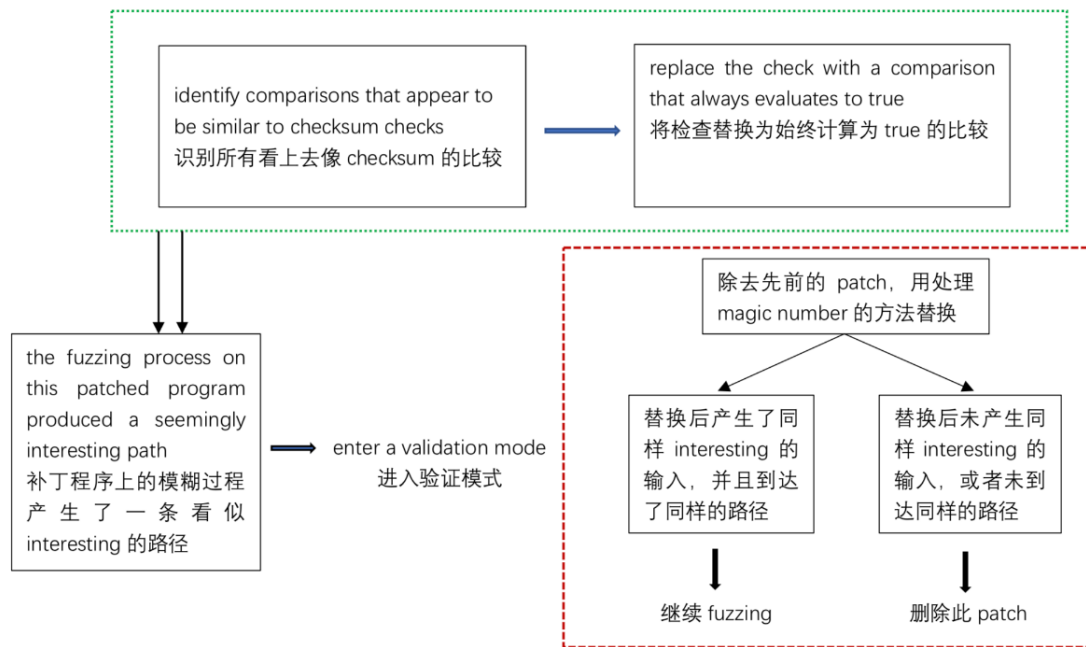
- 对诸如 memcpy 和 strcmp 的比较函数进行处理，hook 所有函数调用
- 如果该函数至少使用两个指针参数，则我们提取指向的前128个字节，以整数的形式处理
- 我们对内存内容使用的编码集与对整数使用的编码集不同
- 要么仅比较前 $n \in \{4, 5, \dots, 32\}$ 个字节（对类似于 memcmp 的函数），要么是直到第一个空字节的所有字节（类似 strcmp 系列的函数）

特定输入字典

- 将包含许多连续的非零或非 0xFF 字节的值添加到字典
- 找到的字符串仅在 havoc 阶段使用

处理校验和

```
if (u64(input) == sum(input+8, len-8))  
    if (u64(input+8) == sum(input+16, len-16))  
        if (input[16] == 'R' && input[17] == 'Q')  
            bug(2);
```



识别校验和的规则

- 在绕过 magic bytes 的过程中找到所有变异规则左侧的 pattern
- 比较的两个参数都不是立即数
- 在着色时，随着输入的变化，pattern 对应的 repl 也在变化。换句话说，比较值取决于输入值

打补丁

- 确定一组可疑的校验和检查后，使用具有成功跳转路径的补丁替换原指令

验证

- 将fuzzing得到的新输入在没有打补丁的真实程序上用之前的魔数字节部分的方式进行修正
- 如果新输入仍然触发新的覆盖，则保留
- 否则，我们将丢弃该补丁

效果

- 在 GNU binutils 上，所有情况下 REDQUEEN 都能比现有工具覆盖更多的代码，与 VUZZER 和 KLEE 相比可将速度提高 5 倍至 5000 倍，对 AFLFAST 和 LAF-INTEL 则可将速度提高 2 倍至 200 倍
- 第一个在 LAVA-M 数据集上发现超过列出错误（2265）的错误（2600），发现了额外的 335 个 BUG，仅错过了列出的 2 个 BUG
- 在 2 个不同的 Linux 文件系统驱动程序中发现 10 个 BUG，在 16 个用户空间程序和软件库中发现了 55 个 BUG
- 获得了 16 个 CVE

AFL局限性

```
1 if (CMP9(ptr, '<', '!', 'A', 'T', 'T', 'L', 'I', 'S', 'T')) {
2     ptr += 9;
3     /* some processing code omitted */
4     while ((ptr != '>') && (ptr != EOF)){
5         int type = 0;
6         if (CMP5(ptr, 'C', 'D', 'A', 'T', 'A')){
7             ptr += 5;
8             type = XML_ATTRIBUTE_CDATA;
9         } else if (CMP6(ptr, 'I', 'D', 'R', 'E', 'F', 'S')){
10             ptr += 6;
11             type = XML_ATTRIBUTE_IDREFS;
12         } else if (CMP5(ptr, 'I', 'D', 'R', 'E', 'F')){
13             ptr += 5;
14             type = XML_ATTRIBUTE_IDREF;
15         } else if ((ptr == 'I') && ((ptr+1) == 'D')){
16             ptr += 2;
17             type = XML_ATTRIBUTE_ID;
18         } else if (CMP6(ptr, 'E', 'N', 'T', 'I', 'T', 'Y')){
19             ptr += 6;
20             type = XML_ATTRIBUTE_ENTITY;
21         } else if (CMP8(ptr, 'E', 'N', 'T', 'I', 'T', 'I', 'E', 'S')){
22             ptr += 8;
23             type = XML_ATTRIBUTE_ENTITIES;
24         } else if (CMP8(ptr, 'N', 'M', 'T', 'O', 'K', 'E', 'N', 'S')){
25             ptr += 8;
26             type = XML_ATTRIBUTE_NMTOKENS;
```

```
27     } else if (CMP7(ptr, 'N', 'M', 'T', 'O', 'K', 'E', 'N')){
28         ptr += 7;
29         type = XML_ATTRIBUTE_NMTOKEN;
30     }
31     if (type == 0) {ptr++; break;}
32
33     /* more omitted code */
34
35     if (CMP9(ptr, '#', 'R', 'E', 'Q', 'U', 'I', 'R', 'E', 'D')) {
36         ptr += 9;
37         default_decl = XML_ATTRIBUTE_REQUIRED;
38     }
39     if (CMP8(ptr, '#', 'I', 'M', 'P', 'L', 'I', 'E', 'D')) {
40         ptr += 8;
41         default_decl = XML_ATTRIBUTE IMPLIED;
42     }
43     if (CMP6(ptr, '#', 'F', 'I', 'X', 'E', 'D')) {
44         ptr += 6;
45         default_decl = XML_ATTRIBUTE_FIXED;
46         if (!IS_BLANK_CH(ptr)) {
47             xmlFatalErrorMsg("Space required after '#FIXED'");
48         }
49     }
50     ptr++;
51 }
52 }
```

观察

- 某些分支只会被极少数 AFL 生成的输入命中（即他们是稀有分支），例如示例的第 1 行
- 稀有分支保护的代码被彻底探索的可能性要远小于被大量输入命中的分支保护的代码

思路

- 识别稀有分支——跟踪命中每个分支的输入数量
- 修改输入变异策略，不允许对针对稀有分支的输入部分进行变异——变异掩码

定义 (1)

- $\text{satisfes}(x, T)$: 如果输入 x 能够满足目标 T (在本论文中可理解为目标程序) , 则 $\text{satisfes}(x, T)$ 为真
- $\mu(c, m)$: 变异方式 , m 为受影响的字节数 , $c \in \{ 0, I, D \}$
- $\text{mutate}(x, \mu, i)$: 在原始输入 x 的位置 i 处施加变异 μ 产生的输入
- $\text{mask}_{x, T}$: 变异掩码 , 返回 $\{ 0, I, D \}$ 的子集。如果 $\text{satisfes}(\text{mutate}(x, (c, 1), i), T)$, 则 $c \in \text{mask}_{x, T}(i)$
- $\text{OKTOMUTATE}(\text{mask}_{x, T}, \mu, k)$:

$$\text{OKTOMUTATE}(\text{mask}_{x, T}, \mu, k) = \bigwedge_{i=k}^{k+m-1} c \in \text{mask}_{x, T}(i).$$

计算变异掩码

Algorithm 3 Computing the mutation mask in FAIRFUZZ.

```
1: procedure COMPUTEMASK(Prog, input, branch)
2:   mask  $\leftarrow$  INITWITHEMPTYSET(|input|)
3:   for  $0 \leq i < |input|$  do
4:     inputO  $\leftarrow$  MUTATE(input, flipByte, i)
5:     if branch  $\in$  BRANCHESHITBY(Prog, inputO) then
6:       mask[i]  $\leftarrow$  mask[i]  $\cup$  {O}
7:     inputI  $\leftarrow$  MUTATE(input, addRandomByte, i)
8:     if branch  $\in$  BRANCHESHITBY(Prog, inputI) then
9:       mask[i]  $\leftarrow$  mask[i]  $\cup$  {I}
10:    inputD  $\leftarrow$  MUTATE(input, deleteByte, i)
11:    if branch  $\in$  BRANCHESHITBY(Prog, inputD) then
12:      mask[i]  $\leftarrow$  mask[i]  $\cup$  {D}
  return mask
```

样例输入 <!ATTLIST ...

- 给定稀有分支 b 为 $L1 \rightarrow L2$, 令 $R = \{O, I, D\}$
- $\text{mask}_{x, b}(0) = \emptyset$
- $\text{mask}_{x, b}(8) = \{I\}$
- $\text{mask}_{x, b}(9) = \{O, I, D\}$
- $\text{OKTOMUTATE}(\text{mask}_{x, b}, (R, 1), 0) = \emptyset$
- $\text{OKTOMUTATE}(\text{mask}_{x, b}, (R, 1), 8) = \emptyset$
- $\text{OKTOMUTATE}(\text{mask}_{x, b}, (R, 1), 8) = I$
- $\text{OKTOMUTATE}(\text{mask}_{x, b}, (R, 1), 9) = \{O, I, D\}$

定义 (2)

- $\text{hits}(x, b)$: 输入为 x 时至少执行分支 b 一次, 则称输入 x 命中分支 b , 表示为 $\text{hits}(x, b)$
- 分支 b 的命中计数 $\text{numHits}[b] = | \{ x \in I : \text{hits}(x, b) \} |$, I 为目前为止 fuzzing 产生的所有输入的集合
- $B_v = \{ b \in B \mid \text{numHits}[b] > 0 \}$, 其中 B 是程序中所有分支的集合
- 稀有分支 b 满足 $\text{numHits}[b] \leq \text{rarity_cutoff}$, 其中 $\text{rarity_cutoff} = 2^i$ (i 满足 $2^{i-1} < \min_{b' \in B_v} (\text{numHits}[b']) \leq 2^i$)
- 最稀有分支 $b^* = \operatorname{argmin}_{b \in \text{branches}(x)} \text{numHits}[b]$

使用变异掩码偏置变异

Algorithm 1 AFL algorithm.

```

1: procedure FUZZTEST(Prog, Seeds)
2:   Queue  $\leftarrow$  Seeds
3:   while true do                                 $\triangleright$  begin a queue cycle
4:     for input in Queue do
5:       if  $\neg$ ISWORTHFUZZING(input) then
6:         continue
7:       score  $\leftarrow$  PERFORMANCESCORE(Prog, input)
8:       for  $0 \leq i < |input|$  do
9:         for mutation in deterministicMutationTypes do
10:          newinput  $\leftarrow$  MUTATE(input, mutation, i)
11:          RUNANDSAVE(Prog, newinput, Queue)
12:       for  $0 \leq i < score$  do
13:         newinput  $\leftarrow$  MUTATEHAVOC(input)
14:         RUNANDSAVE(Prog, newinput, Queue)
15: procedure MUTATEHAVOC(Prog, input)
16:   numMutations  $\leftarrow$  RANDOMBETWEEN(1,256)
17:   newinput  $\leftarrow$  input
18:   for  $0 \leq i < numMutations$  do
19:     mutation  $\leftarrow$  RANDOMMUTATIONTYPE
20:     position  $\leftarrow$  RANDOMBETWEEN(0,  $|newinput|$ )
21:     newinput  $\leftarrow$  MUTATE(newinput, mutation, position)
22:   return newinput
23: procedure RUNANDSAVE(Prog, input, Queue)
24:   runResults  $\leftarrow$  RUN(Prog, input)
25:   if NEWCOVERAGE(runResults) then
26:     ADDTOQUEUE(input, Queue)

```

Algorithm 2 FAIRFUZZ algorithm. Differences from the AFL algorithm are highlighted in gray.

```

1: procedure FAIRFUZZ(Prog, Seeds)
2:   Queue  $\leftarrow$  Seeds
3:   while true do                                 $\triangleright$  begin a queue cycle
4:     for input in Queue do
5:       rarestBranch  $\leftarrow$  RARESTHITBY(Prog, input, numHits)
6:       if numHits[rarestBranch] > rarity_cutoff then
7:         continue
8:       score  $\leftarrow$  PERFORMANCESCORE(Prog, input)
9:       mask  $\leftarrow$  COMPUTEMASK(Prog, input, rarestBranch)
10:      for  $0 \leq i < |input|$  do
11:        for mutation in deterministicMutationTypes do
12:          if  $\neg$ OKTOMUTATE(mask, mutation, i) then
13:            continue
14:          newinput  $\leftarrow$  MUTATE(input, mutation, i)
15:          RUNANDSAVE(Prog, newinput, Queue)
16:        for  $0 \leq i < score$  do
17:          newinput  $\leftarrow$  MUTATEHAVOC(input)
18:          RUNANDSAVE(Prog, newinput, Queue)
19: procedure MUTATEHAVOC(Prog, input)
20:   numMutations  $\leftarrow$  RANDOMBETWEEN(1,256)
21:   newinput  $\leftarrow$  input
22:   for  $0 \leq i < numMutations$  do
23:     mutation  $\leftarrow$  RANDOMMUTATIONTYPE
24:     position  $\leftarrow$  RANDOMOKTOMUTATE(mask, mutation)
25:     newinput  $\leftarrow$  MUTATE(newinput, mutation, position)
26:   return newinput
27: procedure RUNANDSAVE(Prog, input, Queue)
28:   runResults  $\leftarrow$  RUN(Prog, input)
29:   for branch in runResults do
30:     numHits[branch]  $\pm=$  1
31:   if NEWCOVERAGE(runResults) then
32:     ADDTOQUEUE(input, Queue)

```

使用变异掩码偏置变异

- 在havoc阶段，FairFuzz像AFL一样通过选择随机变异类型和随机变异位置来创建变异 μ
- 不是如AFL在0到 $|newinput|-m-1$ 之间随机选择位置，FairFuzz从确定变异位置的子集中随机选择位置

$sampleUniform(\{i \in [0, |x|-m-1] : OKToMUTATE(mask_x, T, \mu, i)\})$

Algorithm 2 FAIRFUZZ algorithm. Differences from the AFL algorithm are highlighted in gray.

```

1: procedure FAIRFUZZ(Prog, Seeds)
2:   Queue  $\leftarrow$  Seeds
3:   while true do ▷ begin a queue cycle
4:     for input in Queue do
5:       rarestBranch  $\leftarrow$  RARESTHitBy(Prog, input, numHits)
6:       if numHits[rarestBranch] > rarity_cutoff then
7:         continue
8:       score  $\leftarrow$  PERFORMANCESCORE(Prog, input)
9:       mask  $\leftarrow$  COMPUTEMASK(Prog, input, rarestBranch)
10:      for  $0 \leq i < |input|$  do
11:        for mutation in deterministicMutationTypes do
12:          if  $\neg OKToMUTATE(mask, mutation, i)$  then
13:            continue
14:          newinput  $\leftarrow$  MUTATE(input, mutation, i)
15:          RUNANDSAVE(Prog, newinput, Queue)
16:          for  $0 \leq i < score$  do
17:            newinput  $\leftarrow$  MUTATEHAVOC(input)
18:            RUNANDSAVE(Prog, newinput, Queue)
19:      procedure MUTATEHAVOC(Prog, input)
20:        numMutations  $\leftarrow$  RANDOMBETWEEN(1, 256)
21:        newinput  $\leftarrow$  input
22:        for  $0 \leq i < numMutations$  do
23:          mutation  $\leftarrow$  RANDOMMUTATIONTYPE
24:          position  $\leftarrow$  RANDOMOKToMUTATE(mask, mutation)
25:          newinput  $\leftarrow$  MUTATE(newinput, mutation, position)
26:        return newinput
27:      procedure RUNANDSAVE(Prog, input, Queue)
28:        runResults  $\leftarrow$  RUN(Prog, input)
29:        for branch in runResults do
30:          numHits[branch]  $\mathrel{+}= 1$ 
31:          if NEWCOVERAGE(runResults) then
32:            ADDTOQUEUE(input, Queue)

```

修剪输入

- 在选择要突变的输入时优先考虑短输入
- 在变异前对父输入进行修剪

效果

- 在 tcpdump、readelf、nm、objdump、C++filt、xmllint、mutool draw、jpeg、readpng 九个 benchmark 上与 AFL、FidgetyAFL、AFLFast.new 进行比较
- 在除了C++filt以外的所有benchmark中，FairFuzz均达到最大的覆盖率，且覆盖率增长速度最快

havoc

- afl-fuzz.c line 6046 ~ 6503
- case 0 : 随机翻转某 bit
- case 1 : 随机选取某 byte 设置为随机的 interesting value
- case 2 : 随机选取某 word 和大小端序设置为随机的 interesting value
- case 3 : 随机选取某 dword 和大小端序设置为随机的 interesting value

havoc

- case 4 : 随机选取某 byte 减去一个随机数
- case 5 : 随机选取某 byte 加上一个随机数
- case 6 : 随机选取某 word 和大小端序减去一个随机数
- case 7 : 随机选取某 word 和大小端序加上一个随机数
- case 8 : 随机选取某 dword 和大小端序减去一个随机数
- case 9 : 随机选取某 dword 和大小端序加上一个随机数
- case 10 : 随机选取某 byte 使用 XOR 设置为随机数
- case 11 ~ 12 : 随机删去某段 bytes

havoc

- case 13 : 随机选取某个位置插入一段随机长度内容
- case 14 : 随机选取某个位置替换为一段随机长度内容
- case 15 : 随机选取某个位置替换为自动生成或用户输入的 token
- case 16 : 随机选取某个位置插入自动生成或用户输入的 token

策略更改

- 识别是bmp文件以后对文件在havoc阶段进行策略挑选
- 每次只进行一种变异策略不再随机，观察变异结果。

针对bmp文件的策略选择实验

```
12  #pragma pack(1)
13  struct bmp_header    //这个结构体就是对上面那个图做一个封装。
14  {
15      //bmp header
16      U8  Signatue[2] ;    // B M
17      U32 FileSize ;      //文件大小
18      U16 Reserv1 ;
19      U16 Reserv2 ;
20      U32 FileOffset ;    //文件头偏移量
21
22      //DIB header
23      U32 DIBHeaderSize ; //DIB头大小
24      U32 ImageWidth  ;   //文件宽度
25      U32 ImageHight  ;   //文件高度
26      U16 Planes      ;
27      U16 BPP         ;   //每个相素点的位数
28      U32 Compression ;
29      U32 ImageSize   ;   //图文件大小
30      U32 XPPM ;
31      U32 YPPM ;
32      U32 CCT ;
33      U32 ICC ;
34  };
35
36  #pragma pack()
```

○ 分析文件头数据类型偏好的策略

- 覆盖率太小，使用bmp文件格式测试upx程序

Q&A